# Query Formulation

## Relational Algebra

# Classification of query languages

- For the **RELATIONAL** data model

**FORMAL** languages

- **Relational Algebra**
- Relational Calculus
- Logic programming (**Datalog**)

**"COMMERCIAL"** languages

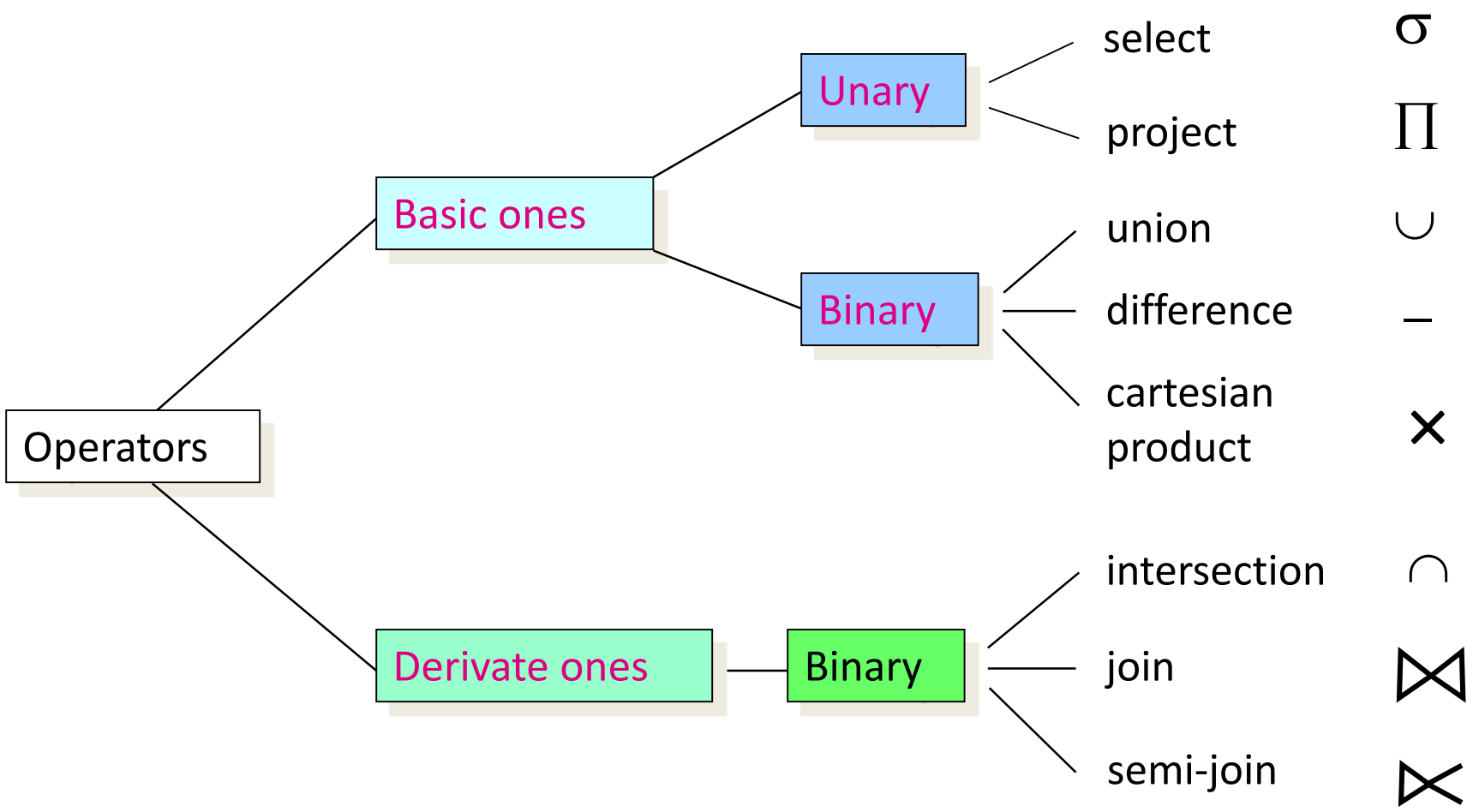
- SQL (Structured Query Language)
- QBE (Query By Example)

# Operators of (our) Relational Algebra

- **Selection**
  - Filters tuples based on a predicate
- **Projection**
  - Prunes out columns that are not interesting (and eliminates duplicates)
- **Redenomination** (and **Assignment**)
  - Give (new) names to relations or to their attributes
- **Cartesian** product
  - Combines the items of two relations in all possible ways
- **Union**, **Difference** and **Intersection**
  - Perform set operations of two relations (*that have the same schema*)

- **Join**: a derived (fundamental) operation
  - Just a name for a Selection over a Cartesian Product

# Global View (taxonomy of the operators)




These operators define a **closed algebra**

(*each operator generates a relation, and expressions are **composable***)

# Dataset

**Student**

| SId | Name | City | Dept |
|---|---|---|---|
| 123 | Carlo | Bologna | CS |
| 415 | Paola | Torino | CS |
| 702 | Antonio | Roma | Log |

**Exam**

| SId | CourseId | Date | Grade |
|---|---|---|---|
| 123 | 1 | 7-9-23 | 30 |
| 123 | 2 | 8-1-24 | 28 |
| 702 | 2 | 7-9-23 | 25 |

**Course**

| CourseId | Title | Teacher |
|---|---|---|
| 1 | Math | Barozzi |
| 2 | Databases | Meo |

# Selection

## $\sigma_{predicate}$ RELATION

generates a relation (with no name) where

- The schema is:
  - the same schema as RELATION
- The instance contains:
  - all tuples of RELATION that satisfy the predicate

Example : $\sigma_{Name='Paola'}$ STUDENT

| SId | Name | City | Dept |
|---|---|---|---|
| 415 | Paola | Torino | CS |

# Selection Predicates

**Predicates are Boolean expressions of Simple predicates**

## Boolean expressions:

| | | |
|---|---|---|
| AND | $(P_1$ AND $P_2)$ | $(\wedge)$ |
| OR | $(P_1$ OR $P_2)$ | $(\vee)$ |
| NOT | $(P_1)$ | $(\neg)$ |

each $P_i$ is a simple predicate

## Simple predicates:

- `true`
- `false`
- ***<term> <comparator> <term>***

***<term>s***:

- Constants
- Attributes
- Arithmetic expressions of terms

***<comparator>s:***

`=  !=  <  <=  >  >=`

$\sigma$ STUDENT

( City='Torino') $\vee$

**(** (City='Roma') $\wedge$ $\neg$ (Dept='Log') **)**

| SId | Name | City | Dept |
|---|---|---|---|
| 123 | Carlo | Bologna | CS |
| 415 | Paola | Torino | CS |
| 702 | Antonio | Roma | Log |

# Projection

$\Pi_{att1, \ldots, attN}$ ***RELATION***

- generates a relation (with no name) where
- the schema:
  - only contains the mentioned attributes
- the instance contains:
  - the restriction of all tuples over the mentioned attributes

$\Pi_{Name,Dept}$ STUDENT

| Name | Dept |
|---|---|
| Carlo | CS |
| Paola | CS |
| Antonio | Log |

# Projection and Duplicates

- In the formal model, the projection eliminates the duplicates

$$\Pi_{\text{Dept}}\,\text{STUDENT}$$

| SId | Name | City | Dept |
|---|---|---|---|
| 123 | Carlo | Bologna | CS |
| 415 | Paola | Torino | CS |
| 702 | Antonio | Roma | Log |

| Dept |
|---|
| CS |
| Log |

- In **real systems** (i.e., in the informal model), the elimination of duplicates MUST be explicitly requested

# Union

## RELATION1 ∪ RELATION2

Generates the union of the two relations

Can be done only if the relations are compatible

Thus, they must have the same schema

On real systems, the constraint is relaxed:

- The same degree
- Their domains have the same types (in order)

# Intersection ( ∩ ) and Difference ( – )

- The definitions are straightforward, with the same restrictions on the schema (attribute names and data types)

# Cartesian Product

## $R \times S$

generates a relation (with no name) where

- The schema has:
  - all attributes from both R and S
  - degree(RxS) = degree(R)+degree(S)
- The instance has:
  - all possible pairs of tuples of R and of S, juxtaposed
  - card(RxS)=card(R)*card(S)

If R and S have attributes with the same name, they are made different and identified by using the table names as prefixes ("**dot notation**")

**R**(A,B,C) **S**(C,D) → RxS( A, B, **R.**C, **S.**C, D )

# Example of Cartesian Product

R(A,B)

| A | B |
|---|---|
| a | 1 |
| b | 3 |

S(C,D)

| C | D |
|---|---|
| c | 1 |
| b | 3 |
| a | 2 |

R × S (A,B,C,D)

| A | B | C | D |
|---|---|---|---|
| a | 1 | c | 1 |
| a | 1 | b | 3 |
| a | 1 | a | 2 |
| b | 3 | c | 1 |
| b | 3 | b | 3 |
| b | 3 | a | 2 |

# Join

$R \bowtie_{predicate} S$

**Defined as:** $\sigma_{predicate}(R \times S)$

generates a relation (with no name) where

- The schema is:
  - The concatenation of the schemas of STUDENT and EXAM
- The instance has:
  - The tuples of the Cartesian product that fulfill the selection predicate

# Join

$$\text{STUDENT} \bowtie_{\text{STUDENT.SId = EXAM.SId}} \text{EXAM}$$

$$\sigma_{\text{STUDENT.SId = EXAM.SId}} (\text{STUDENT} \times \text{EXAM})$$

Attributes with the same name are identified by prefixes, with the "**dot notation**":

EXAM.SId, STUDENT.SId

| **STUDENT**.SId | Name | City | Dept | **EXAM**.SId | CourseId | Date | Grade |
|---|---|---|---|---|---|---|---|
| **123** | Carlo | Bologna | CS | **123** | 1 | 7-9-13 | 30 |
| **123** | Carlo | Bologna | CS | **123** | 2 | 8-1-14 | 28 |
| **702** | Antonio | Roma | Log | **702** | 2 | 7-9-13 | 25 |

# Syntax of the Join **predicate**

**Conjunctive** expressions of simple predicates in the form

$$R \bowtie_{attr1\ comp\ attr2} S$$

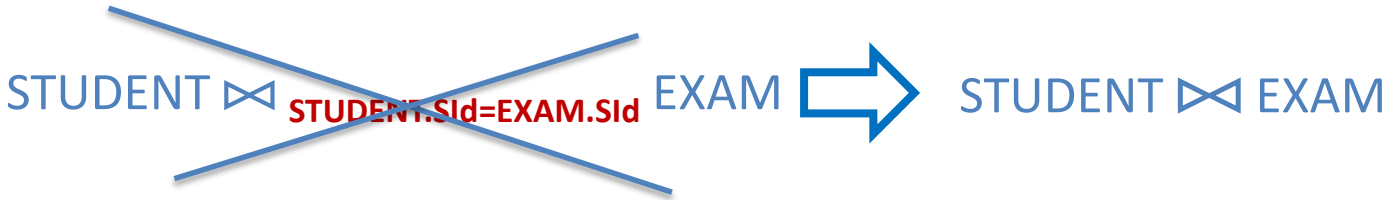
where ***attr1*** belongs to R
***attr2*** belongs to S
***comp*** is one of : **=** != < <= > >=

## Most often, only EQUI-JOINs are considered

Joins whose predicates only have equality comparisons ( **=** )

# A little **trick** (to write things more quickly)

- **Equi-Joins** typically match *corresponding attributes*
  - i.e., attributes expressing the same property
    - e.g., the **SId** of the student in STUDENT and the SId in EXAM
- Corresponding attributes often have the **same name**
  - Attributes that express the same property (e.g., SId in STUDENT and in EXAM, to state that an exam was done by that particular student)
- But this is the **worst case** to write the join query !
  - Requires the use of prefixes with dot notation (or redenomination)
- **Idea:**




  - The **NATURAL join** (*written with no predicate*) matches **all homonymous attributes** without replicating the column
  - **The predicate is IMPLIED !** *(together with a little projection)*

# (natural) Join of Three Tables

## STUDENT ⋈ EXAM ⋈ COURSE

| **SId** | Name | City | Dept | **CourseId** | Date | Grade | Title | Teacher |
|---|---|---|---|---|---|---|---|---|
| **123** | Carlo | Bologna | CS | **1** | 7-9-13 | 30 | Math | Barozzi |
| **123** | Carlo | Bologna | CS | **2** | 8-1-14 | 28 | Databases | Meo |
| **702** | Antonio | Roma | Log | **2** | 7-9-13 | 25 | Databases | Meo |

***Equivalent to computing either***

**( STUDENT ⋈ EXAM ) ⋈ COURSE**

***or***

**STUDENT ⋈ ( EXAM ⋈ COURSE )**

# Redenomination (and Assignment)

- Assignment ( := )
  - Provides the resulting relations with a name
  - Not exactly an algebraic operator

**CScientists** := $\sigma_{\text{Dept='CS'}}$ STUDENT

**Turiners** := $\sigma_{\text{City='Torino'}}$ STUDENT

- Ridenomination ( $\rho$ )
  - Changes the name of some attributes

$\rho_{\text{PlaceOfOfBirth} \leftarrow \text{City}}$ STUDENT

  - This is very useful (actually necessary) when joining a relation… with itself! Otherwise the "dot notation" is insufficient to express a join predicate without ambiguity, because for attributes with the same name also the prefixes would be the same:

EXAM $\bowtie_{\text{SId=SId}}$ EXAM → ~~EXAM $\bowtie_{\text{EXAM.SId=EXAM.SId}}$ EXAM~~ → EXAM $\bowtie_{\text{SId=Id2}}$ $\rho_{\text{Id2} \leftarrow \text{SId}}$ EXAM

# More on redenomination and Assignment

PATERNITY(Father, Child)

MATERNITY(Mother, Child) → MATERNITY ∪ PATERNITY ?

We simply change the attribute names

$\rho_{Parent \leftarrow Father}$ **PATERNITY**

$\rho_{Parent \leftarrow Mother}$ **MATERNITY**

**PARENTSHIP** := $\rho_{Parent \leftarrow Father}$ **PATERNITY** ∪ $\rho_{Parent \leftarrow Mother}$ **MATERNITY**

***Useful to «force» (algebraic) compatibility between formally different domains***

# Equivalence of Expressions

Find the names of the students who got a 30 in Math

$\Pi_{\text{Name}}\ \sigma_{\text{Grade=30} \wedge \text{Title='Math'}}\ (\text{STUDENT} \bowtie \text{EXAM} \bowtie \text{COURSE})$

*Is this the only way to express the query?*

It is equivalent to

$\Pi_{\text{Name}}\ (\text{STUDENT} \bowtie (\sigma_{\text{Grade=30}}\ \text{EXAM} \bowtie (\sigma_{\text{Title='Math'}}\ \text{COURSE})))$

# Equivalence of Expressions

**1**. Antonio's teachers?

$\Pi_{\text{Teacher}}\ \sigma_{\text{Name = 'Antonio'}}$ ( STUDENT $\bowtie$ EXAM $\bowtie$ COURSE )

Equivalent to:

$\Pi_{\text{Teacher}}$ ( COURSE $\bowtie$ ( EXAM $\bowtie$ $\sigma_{\text{Name = 'Antonio'}}$ STUDENT ) )

# Examples

**2a**. Find the **SIds** of the students who passed at least **one** exam

$$\Pi_{SId}\ EXAM$$

**2b**. Find the **names** of those who passed at least **one** exam

$$\Pi_{Name}\ (\ STUDENT \bowtie EXAM\ )$$

**3**. Find the **names** of those who passed at least **two** exams

$$\Pi_{Name}\ (\ STUDENT \bowtie (\ EXAM \bowtie_{SId = SId' \wedge CourseId \mathrel{!}= CourseId'} EXAM'\ )\ )$$

Here SId', CourseId' and EXAM' are used instead of (trivial) redenomination/assignment:

$$EXAM' := \rho_{SId',CourseId',Date',Grade' \leftarrow SId,CourseId,Date,Grade}\ EXAM$$

$$EXAM \bowtie_{SId = SId' \wedge CourseId \mathrel{!}= CourseId'}\ EXAM'$$

*Also equivalent to writing :* $EXAM \triangleright\!\triangleleft_{SId=X \wedge CourseId \mathrel{!}= Y}\ \rho_{X,Y \leftarrow SId,CourseId}\ EXAM$

# Examples (*a digression on cardinalities*)

**3**. Find the **names** of those who passed at least **two** exams (**again!**)

(*If we replace* **!=** *with* **>** *the intermediate results are reduced by half in size*)

$$\Pi_{\text{Name}} ( \text{STUDENT} \bowtie ( \text{EXAM} \bowtie_{\text{SId} = \text{SId'} \wedge \text{CourseId} > \text{CourseId'}} \text{EXAM'} ) )$$

**3bis**. Find the **names** of those who passed at least **three** exams

$$\Pi_{\text{Name}} ( \text{STUDENT} \bowtie ( \text{EXAM} \bowtie_{\text{SId}=\text{SId'} \wedge \text{CourseId} > \text{CourseId'}} \text{EXAM'} \bowtie_{\text{SId'}=\text{SId''} \wedge \text{CourseId'} > \text{CourseId''}} \text{EXAM''} ) )$$

**3ter**. Find the **names** of those who passed at least **four** exams

$$\Pi_{\text{Name}} ( \text{STUDENT} \bowtie ( \text{EXAM} \bowtie_{\text{SId}=\text{SId'} \wedge \text{CrsId} > \text{CrsId'}} \text{EXAM'} \bowtie_{\text{SId'}=\text{SId''} \wedge \text{CrsId'} > \text{CrsId''}} \text{EXAM''} \bowtie_{\text{SId''}=\text{SId'''} \wedge \text{CrsId''} > \text{CrsId'''}} \text{EXAM'''} ) )$$

# Examples (*a digression on cardinalities*)

**3quater**. Find the **names** of those who passed **exactly three** exams

*Chains of joins only allow to express cardinality conditions in the form of "at least n" occurrences of some fact – namely in our examples we can join n versions of the same EXAM table so as to match n distinct exams by the same user, always identified by the same SId.*
*However, if we think of the results of queries 3bis and 3ter, we realize that the only students who are included in the former and not in the latter are… just those wo passed 3 exams and no more than 4 exams* → **exactly three** **!**

**We can therefore express our query by subtracting the latters from the formers!**

$$\Pi_{\text{Name}} \Big( \text{STUDENT} \bowtie \Big( \Pi_{\text{SId}} \big( \text{EXAM} \bowtie_{\text{SId}=\text{SId}' \wedge \text{CrsId} > \text{CrsId}'} \text{EXAM}' \bowtie_{\text{SId}'=\text{SId}'' \wedge \text{CrsId}' > \text{CrseId}''} \text{EXAM}'' \big) - \Pi_{\text{SId}} \big( \text{EXAM} \bowtie_{\text{SId}=\text{SId}' \wedge \text{CrsId} > \text{CrsId}'} \text{EXAM}' \bowtie_{\text{SId}'=\text{SId}'' \wedge \text{CrsId}' > \text{CrsId}''} \text{EXAM}'' \bowtie_{\text{SId}''=\text{SId}''' \wedge \text{CrsId}'' > \text{CrsId}'''} \text{EXAM}''' \big) \Big) \Big)$$

# Examples

**4**. Find the name of the students who **never** passed ***any*** exam

$$\Pi_{\text{Name}} ( \text{STUDENT} \bowtie ( \Pi_{\text{SId}} \text{STUDENT} - \Pi_{\text{SId}} \text{EXAM} ) )$$

**5**. Find the names of who **never** passed ***the Math*** exam

$$\Pi_{\text{Name}} ( \text{STUDENT} \bowtie ( \Pi_{\text{SId}} \text{STUDENT} - \Pi_{\text{SId}} ( \text{EXAM} \bowtie (\sigma_{\text{Title='Math'}} \text{COURSE}) ) ) )$$

# Complex Expressions

**6**. Names of students who **always** got more than 28 as grade

$$\Pi_{\text{Name}} \Big( \text{STUDENT} \bowtie \big( \Pi_{\text{SId}} \text{ EXAM} - \Pi_{\text{SId}} \, \sigma_{\text{Grade <= 28}} \text{ EXAM} \big) \Big)$$

Rationale:

- First, find the SIds of those who passed at least one exam
- Then, subtract the SIds of those who, at least once, got a 28 or less
  - The remaining SIds are those of the students who never «failed» , i.e. those who were always excellent (29 or 30)
- Finally, join those SIds with STUDENT to extract their names

# Complex Expressions

**6bis**. The names of all the students who **either** always got more than 28 **or** never passed any examination

$$\Pi_{\text{Name}}\ \text{STUDENT} \bowtie \mathbf{(}$$
$$(\ \Pi_{\text{SId}}\ \text{EXAM} - \Pi_{\text{SId}}\ \sigma_{\text{Grade <= 28}}\ \text{EXAM}\ )$$
$$\cup$$
$$(\ \Pi_{\text{SId}}\ \text{STUDENT} - \Pi_{\text{SId}}\ \text{EXAM}\ )$$
$$\mathbf{)}$$

# *Equivalence of expressions (a tricky one)*

**6bis**. The names of all the students who **either** always got more than 28 **or** never passed any examination

***Is this variant equivalent?***

$$( \Pi_{\text{Name}} \text{ STUDENT} \bowtie ( \Pi_{\text{SId}} \text{ EXAM} - \Pi_{\text{SId}} \sigma_{\text{Grade <= 28}} \text{EXAM} ) )$$

$$\cup$$

$$( \Pi_{\mathbf{Name}} \text{ STUDENT} - \Pi_{\mathbf{Name}} ( \text{STUDENT} \bowtie \text{EXAM} ) )$$

# Complex Expressions

**7**. Find the names of the students who passed “Databases” and “Math” **on the same day**

$$\Pi_{\text{Name}}\ \text{STUDENT} \bowtie \Big( \big( \text{EXAM} \bowtie \sigma_{\text{Title = “Databases"}}\ \text{COURSE} \big) \bowtie_{\text{SId=SId'} \wedge \textbf{Date = Date'}} \big( \text{EXAM'} \bowtie \sigma_{\text{Title = "Math"}}\ \text{COURSE} \big) \Big)$$

Rationale:

- first, separately list the exams of Databases and Math
- then, join the two lists based on identity of student **and date**
  - thus finding the SIds, but not yet the Names
- Finally, join with STUDENT to extract their names

# Complex Expressions

**8**. Find the **first** exam **ever**

***hint**: all exams have a previous exam… but the **first** ones*

EXAM

–

$$\Pi_{SId,CourseId,Date,Grade}( EXAM \bowtie_{Date > Dat2} (\rho_{SId2,CId2,\mathbf{Dat2},Gr2 \leftarrow SId,CourseId,\mathbf{Date},Grade} EXAM) )$$

Rationale:

- first, find all the exams that are “not first ones”
  - (i.e., exams that are preceded by another exam)
- Then, subtract this set from the set of all the exams
- The remainder is**(are)** the first exam**(s)**
  - More likely: all the exams of the first examination day

# Complex Expressions

**9**. Find the **first** exam **of each student**

***hint**: all exams by user **u** have a previous exam by user **u**… but one!*

$$\text{EXAM} - \Pi_{\text{SId,CourseId,Date,Grade}}( \text{EXAM} \bowtie_{\text{Date} > \text{Dat2} \wedge \textbf{SId} = \textbf{SId2}} ( \rho_{\textbf{SId2},\text{CId2},\textbf{Dat2},\text{Gr2} \leftarrow \textbf{SId},\text{CourseId},\textbf{Date},\text{Grade}} \text{EXAM} ) )$$

Rationale:

- First, find all the exams that are "not first ones" for each user
  - (i.e., exams that are preceded by another exam **by the same user**)
- Then, subtract this set to the set of all the exams
- The remainder is**(are)** the first exam**(s) of each user**
  - Now it is unlikely that **u** passed her first two exams on the same day

# Analysis of other schemas

SUPPLIER ( SuppCode, Name, City, Address, Phone )

CATALOGUE ( SuppCode, ProdCode, UnitaryCost )

PRODUCT ( ProdCode, Name, Brand, Model, Version, Descritpion )

*Queries:*

*Is there any product that is supplied by one supplier only? (exclusive)*

*Is there any product that is supplied by 3 or more suppliers, but the Cost is actually the same for them all?*

*Is there any «universal» supplier, i.e. a single supplier offering all the products that we sell? (possibly not all at the most convenient price)*

# Analysis of other schemas

CLIENT ( SSN, Name, Address, Phone )

RENTAL ( PlateN, SSN, DateStart, DateEnd )

CAR ( PlateN, Model, Color, MatrYear, Kms )

MODEL ( ModName, Manuf, ReleaseYear, Displacement,
Segment, MaxSpeed )

*Queries:*

*What's the fastest car rented by each client?*

*Is there any car that was never re-taken by any of the clients who rented it once?*

*Is there any client who always rented cars of a single model only?*

# Analysis of other schemas

DIRECTOR ( Name, Birthdate, Nationality )

ACTOR ( Name, Birthdate, Nationality )

ROLE ( ActorN, Title, Character )

MOVIE ( Title, DirectorN, Year, Duration, Genre)

SHOWING ( CityTh, NameTh, Title )

THEATRE ( City, Name, Address, #Rooms, #Seats )

*Actors who played two or more distinct characters in the same movie?*

*Actors who played the same character in two distinct movies?*

*Is there any director who played at least one (possibly very small) one role in all the movies that they directed? [Hitchcock? Tarantino?]*

*Is there any «versatile» actor who played in at least one movie of every Genre known to the database?*

# Analysis of other schemas

USER ( UsrId, Name, Email, Photo, SubscriptionDate )

FRIENDS ( UsrId, FriendId, RestrictedList )

POST ( PostId, UsrId, Date, Time, Text, PublicationLocation,
VisibilityLevel, TimestampLastEdit )

REACTION ( PostId, UrsId, ReactionType )

RestrictedList is a flag (yes/no) used to distinguish the friends within the closest circle (as opposed to generic friends).

Friendship is necessarily symmetric, and the relation explicitly contains distinct tuples to represent the relationship in both directions (Id1→Id2 and Id2→Id1), but being in the restricted list isn't a symmetric property.

Each user can express a reaction to any post, (even to their own posts!) and ReactionType can be only 'LikeIt', 'LoveIt', 'AhAh', 'Wow', 'Sigh' e 'Grrr'